

Paradigma Funcional: Evaluación, Abstracciones y Mecanismos de Aplicación

Resumen Ejecutivo

El presente documento sintetiza los conceptos fundamentales del paradigma funcional abordados en la Unidad 4 de la cátedra de Programación II de la Facultad de Tecnología y Ciencias Aplicadas (Plan 2011, Año 2025). El análisis se centra en el proceso de evaluación de expresiones, comparando las estrategias ansiosa y perezosa, y profundiza en herramientas de abstracción avanzadas como las expresiones lambda y el polimorfismo. Asimismo, se examina el mecanismo de parcialización (*currying*), que permite la aplicación parcial de funciones y la creación de secciones de operadores, concluyendo con los fundamentos matemáticos y técnicos de la composición de funciones. Los puntos clave incluyen la eficiencia operativa de la evaluación perezosa y la capacidad de las funciones currificadas para generar dinámicamente nuevos comportamientos computacionales.

1. Evaluación de Expresiones

La evaluación en el paradigma funcional se define como el proceso de transformar una expresión aplicando definiciones de funciones de manera iterativa hasta alcanzar una forma que no admite más transformaciones, denominada **representación canónica**.

Existen dos trayectorias o esquemas principales para este proceso:

1.1. Evaluación Ansiosa (Llamada por Valor)

- **Mecánica:** Se evalúan primero las expresiones internas (argumentos) antes de la aplicación de la función principal.
- **Riesgos:** Puede derivar en bucles infinitos en casos donde los argumentos no tienen una forma normal, incluso si la función no requiere dicho argumento para devolver un resultado.
- **Ejemplo:** En la expresión `tres infinito`, donde `infinito = infinito + 1`, la evaluación ansiosa intentará resolver `infinito` indefinidamente, fallando en retornar el valor `3`.

1.2. Evaluación Perezosa (Llamada por Nombre)

- **Mecánica:** Utiliza la definición de la función antes de evaluar los argumentos.
 - **Ventajas:**
 - **Normalizadora:** Siempre encuentra la forma normal de una expresión si esta existe.
 - **Trabajo Mínimo:** Solo reduce los *redexes* (expresiones reducibles) estrictamente necesarios para calcular el valor final.
 - **Ejemplo:** En la misma expresión `tres infinito`, bajo evaluación perezosa se aplica primero la definición de la función `tres x = 3`, obteniendo el resultado inmediatamente sin evaluar el argumento infinito.
-

2. Expresiones Lambda

Además de las funciones con nombre, el paradigma permite definir funciones anónimas mediante **expresiones lambda**.

- **Sintaxis:** `\ <patrones> -> <expr>`
 - **Funcionamiento:** Denota una función que recibe un número de parámetros (uno por patrón) y produce el resultado definido en `<expr>`.
 - **Ejemplos técnicos:**
 - Cuadrado de un número: `(\ x -> x * x)`
 - Suma de dos argumentos: `(\ x y -> x + y)`
-

3. Polimorfismo

El polimorfismo permite que ciertas funciones operen con argumentos de diversos tipos de datos, utilizando variables de tipo (usualmente representadas por la letra `a`).

Función	Definición de Tipo	Descripción
<code>espejo</code>	<code>a -> a</code>	Devuelve el mismo elemento recibido, sin importar su tipo.

<code>longz</code>	<code>[a] -> Integer</code>	Calcula la longitud de una lista de cualquier tipo de elementos.
--------------------	--------------------------------	--

4. Mecanismo de Parcialización (*Currying*)

La parcialización es un proceso fundamental donde las funciones de múltiples argumentos se interpretan como una secuencia de funciones que toman un solo argumento y devuelven otra función.

4.1. Definiciones Equivalentes

Una función como `sumaCuadrados x y = x*x + y*y` puede expresarse formalmente mediante abstracciones lambda anidadas: `sumaCuadrados = \ x -> ( \ y -> x*x + y*y )`

4.2. Reglas de Asociatividad

Para el correcto funcionamiento de este mecanismo, se aplican las siguientes reglas:

1. **En los tipos:** El constructor `->` es asociativo a la **derecha**.
 - `Int -> Int -> Int` equivale a `Int -> (Int -> Int)`.
2. **En la aplicación:** La aplicación de funciones es asociativa a la **izquierda**.
 - `mult x y z` equivale a `((mult x) y) z`.

4.3. Aplicación Parcial

Permite aplicar a una función menos argumentos de los que define su signatura, resultando en una nueva función "precargada".

- **Caso práctico:** Si definimos `multiploDe p n = n mod p == 0`, podemos crear la función `esPar` mediante aplicación parcial: `esPar = multiploDe 2`.

5. Secciones de Operadores

Las secciones son una forma particular de aplicación parcial aplicada a operadores binarios (como `+`, `*`, `^`).

- **Tipos de secciones:**

- $(x \otimes)$: Función que toma un argumento y lo sitúa a la derecha del operador.
- $(\otimes y)$: Función que toma un argumento y lo sitúa a la izquierda del operador.
- $(\otimes)$: El operador como función de dos argumentos.

Nota sobre conmutatividad: En operadores como la suma $(+)$, las secciones $(2+)$ y $(+2)$ son equivalentes. Sin embargo, en operadores no conmutativos como la potencia $(^)$, $(2^)$ (potencia de base 2) es distinto de $(^2)$ (elevar al cuadrado).

6. Composición de Funciones

La composición permite crear una nueva función combinando dos funciones existentes, de modo que el resultado de la primera sea el argumento de la segunda.

6.1. Definición Formal

En Haskell, se utiliza el operador punto $(.)$: `infixr 9 ( . ) :: (b -> c) -> (a -> b) -> a -> c` Matemáticamente: $(g \circ f) x = g(f(x))$

6.2. Restricciones de Composición

Para que dos funciones g y f sean componibles como $g . f$, deben cumplirse tres condiciones:

1. El tipo de resultado de f debe coincidir con el tipo de argumento de g .
2. El tipo de resultado de la composición $(g . f)$ es el mismo que el resultado de g .
3. El tipo del argumento de la composición $(g . f)$ es el mismo que el tipo del argumento de f .

6.3. Ejemplo de Aplicación

Para determinar si la longitud del nombre de una persona es par, se pueden componer las funciones `esPar` y `longitud`:

- `nombrePar = esPar . longitud`
- Flujo: `String` (Nombre) $\rightarrow$ `Int` (Longitud) $\rightarrow$ `Bool` (¿Es par?).